

Workshop2 - 02bash

Parte 1

```
feijo@valenfei MINGW64 ~/02-bash/WS2 (master)
$ cat << 'EOF' > Dockerfile
FROM ubuntu:latest

RUN apt-get update && apt-get install -y \
    bash \
    coreutils \
    grep \
    gawk \
    procs \
    nano \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY . /app

CMD ["bash"]
EOF
```

```
feijo@valenfei MINGW64 ~/02-bash/WS2 (master)
$ docker build -t ubuntu-bash-lab .
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 251B 0.0s done
#1 DONE 0.1s

#2 [internal] load metadata for docker.io/library/ubuntu:latest
#2 DONE 2.4s

#3 [internal] load .dockerignore
#3 transferring context: 2B done
#3 DONE 0.1s

#4 [internal] load build context
#4 transferring context: 251B 0.0s done
#4 DONE 0.1s

#5 [1/4] FROM docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09b
#5 resolve docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09b 0.1s done
#5 sha256:f3db1cd940786339b09d8a60e47c66fea9502d788e6fab5bec91a4a77d4ced1c 393B / 393B 0.2s done
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 0B / 41.57MB 0.2s
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 4.19MB / 41.57MB 0.5s
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 7.34MB / 41.57MB 0.6s
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 10.49MB / 41.57MB 0.8s
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 13.63MB / 41.57MB 0.9s
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 17.83MB / 41.57MB 1.1s
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 22.02MB / 41.57MB 1.2s
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 24.12MB / 41.57MB 1.4s
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 29.36MB / 41.57MB 1.5s
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 32.51MB / 41.57MB 1.7s
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 38.80MB / 41.57MB 1.8s
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 40.89MB / 41.57MB 2.0s
#5 sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 41.57MB / 41.57MB 2.4s done
#5 extracting sha256:06e9d71331fb2b620a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 1.0s done
#5 DONE 3.5s

#5 [1/4] FROM docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09b
#5 extracting sha256:f3db1cd940786339b09d8a60e47c66fea9502d788e6fab5bec91a4a77d4ced1c 0.0s done
#5 DONE 3.5s

#6 [2/4] RUN apt-get update && apt-get install -y bash coreutils grep gawk procs nano && rm -rf /var/lib/apt/lists/*
#6 0.499 Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
#6 0.739 Get:2 http://archive.ubuntu.com/ubuntu resolute InRelease [136 kB]
#6 0.938 Get:3 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [196 kB]
#6 1.169 Get:4 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [444 kB]
#6 1.300 Get:5 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [520 kB]
#6 1.389 Get:6 http://security.ubuntu.com/ubuntu resolute-security/multiverse amd64 Packages [11.2 kB]
#6 1.408 Get:7 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
#6 1.724 Get:8 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
#6 1.993 Get:9 http://archive.ubuntu.com/ubuntu resolute/restricted amd64 Packages [189 kB]
#6 2.423 Get:10 http://archive.ubuntu.com/ubuntu resolute/main amd64 Packages [1874 kB]
```

```

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ winpty docker run -it --name ubuntu-lab ubuntu-bash-lab
root@2e68afe5c62d:/app# cd /app
root@2e68afe5c62d:/app# ls -la
total 12
drwxr-xr-x 1 root root 4096 Aug 30 18:58 .
drwxr-xr-x 1 root root 4096 Aug 30 19:01 ..
-rwxr-xr-x 1 root root 212 Aug 30 18:56 Dockerfile
root@2e68afe5c62d:/app# |

```

Parte 2. Ejercicio CMD vs ENTRYPOINT

Caso A

```

MINGW64/c/Users/feijo/02-bash/WS2
GNU nano 8.2 Dockerfile
FROM ubuntu:latest
RUN apt-get update && apt-get install -y \
    bash \
    coreutils \
    grep \
    gawk \
    procps \
    nano \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY . /app
CMD ["bash"]

```

```

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ nano Dockerfile
feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ cat Dockerfile

FROM ubuntu:latest
CMD ["echo", "Hola desde CMD"]

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$

```

```

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ docker build -t ejemplo-cmd .
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 89B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/ubuntu:latest
#2 DONE 1.5s

#3 [internal] load .dockerignore
#3 transferring context: 2B done
#3 DONE 0.0s

#4 [1/1] FROM docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348
#4 resolve docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2
#4 CACHED

#5 exporting to image
#5 exporting layers done
#5 exporting manifest sha256:3ed3aa4d84c511c9c3bf3da13d378c368476a3276d537aef409d2450993e71d8 0.0s done
#5 exporting config sha256:bf8cb9f21a90fd894a7f4c0b2ca20fd31f39677463bb206038636fffd59132e2e 0.0s done
#5 exporting attestation manifest sha256:435144a28f78bcc5d3d542d5ea0ef66f909ec0eed3417743e7353af1739614
#5 exporting manifest list sha256:7aece4669e5d08d4604621ba883de8751461a8d1b28ccb1332d0b1fef7578c71 0.0s
#5 naming to docker.io/library/ejemplo-cmd:latest done
#5 unpacking to docker.io/library/ejemplo-cmd:latest 0.0s done
#5 DONE 0.2s

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$

```

```

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ docker run --rm ejemplo-cmd
Hola desde CMD

```

```

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ docker run --rm ejemplo-cmd "Mensaje reemplazado"
docker: Error response from daemon: failed to create task for container: failed to create shim
d: runc create failed: unable to start container process: error during container init: exec: "
e file not found in $PATH
Run 'docker run --help' for more information

```

Al ejecutar la imagen sin argumentos, se ejecuta el `CMD` definido en el Dockerfile. Al proporcionar `"Mensaje reemplazado"` al final de `docker run`, Docker reemplaza completamente el `CMD` e intenta ejecutar ese texto como un comando. Como no existe un ejecutable con ese nombre, se produce un error.

Caso B: solo ENTRYPOINT

```

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ nano Dockerfile

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ cat Dockerfile

FROM ubuntu:latest
ENTRYPOINT ["echo"]

```

```

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ docker build -t ejemplo-entrypoint .
docker run --rm ejemplo-entrypoint "Hola"
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 78B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/ubuntu:latest
#2 DONE 0.5s

#3 [internal] load .dockerignore
#3 transferring context: 2B done
#3 DONE 0.0s

#4 [1/1] FROM docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09
#4 resolve docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09
#4 CACHED

#5 exporting to image
#5 exporting layers done
#5 exporting manifest sha256:e98a316fa79fd45fd55f4a89c65093ae3375bbf5403f90fdd0fa105143d67287 0.0s done
#5 exporting config sha256:73228b9a057d67d302bb2bf72144651db07dc5d1bacb047df8d9b8b9c59b636d 0.0s done
#5 exporting attestation manifest sha256:3915d805fc2c583097298f83af6308ecd06358d8b048684637fa71cc224bc58a 0.0s done
#5 exporting attestation manifest sha256:3915d805fc2c583097298f83af6308ecd06358d8b048684637fa71cc224bc58a 0.0s done
#5 exporting manifest list sha256:373b17c932b6dda0226cc478b328ad1bec3bec949ef25d3eea7e0e8b759f33cf 0.0s done
#5 naming to docker.io/library/ejemplo-entrypoint:latest done
#5 unpacking to docker.io/library/ejemplo-entrypoint:latest 0.0s done
#5 DONE 0.1s
Hola
feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)

```

```
feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ docker run --rm ejemplo-entrypoint "Mensaje adicional"
Mensaje adicional
```

Caso C: ENTRYPOINT + CMD

```
feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ nano Dockerfile

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ cat Dockerfile

FROM ubuntu:latest
ENTRYPOINT ["echo"]
CMD ["Mensaje por defecto"]

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ |
```

```
feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ docker build -t ejemplo-combinado .
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 106B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/ubuntu:latest
#2 DONE 1.6s

#3 [internal] load .dockerignore
#3 transferring context: 2B done
#3 DONE 0.0s

#4 [1/1] FROM docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d
#4 resolve docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d
#4 CACHED

#5 exporting to image
#5 exporting layers done
#5 exporting manifest sha256:d4f9e5e34f52fae5eefa05060fd9153d58476371da2d3ed33d5ad0a6f807acfd 0.0s done
#5 exporting config sha256:b139486132307837949646b89890488e6ef915bedf12ece9df6e5c36bf8629a3 0.0s done
#5 exporting attestation manifest sha256:bb6445eab0df19da8c655d63f12c5c754c4eb67801af99408be7d74a71e4ba9c 0
#5 exporting manifest list sha256:eb935325925d02933f37e865e09232f98a772861796ad07e87e245e338fbeeac
#5 exporting manifest list sha256:eb935325925d02933f37e865e09232f98a772861796ad07e87e245e338fbeeac 0.0s don
#5 naming to docker.io/library/ejemplo-combinado:latest done
#5 unpacking to docker.io/library/ejemplo-combinado:latest 0.0s done
#5 DONE 0.2s

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ docker images
IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
docker/welcome-to-docker:latest     c4d56c24da4f                      22.2MB      6.03MB
ejemplo-cmd:latest                   7aece4669e5d                      157MB       41.6MB
ejemplo-combinado:latest             eb935325925d                      157MB       41.6MB
ejemplo-entrypoint:latest           373b17c932b6                      157MB       41.6MB
gcc:latest                           1cfa8769230d                      2.21GB      577MB
japeto/pujgcc:latest                9b3d7bbf4103                      2.18GB      448MB
japeto/so-tools:v64                 7e990f123c8d                      2GB         506MB
japeto/sorting:latest               d9a6607c8dad                      2.17GB      551MB
jupyter/base-notebook:latest        8c903974902b                      1.48GB      311MB
ubuntu-bash-lab:latest              97d068e37d2c                      162MB       43MB      U
val/pyapp:latest                    3311a1fb96ce                      197MB       48.1MB
```

```
MINGW64:/c/Users/feijo/02-bash/WS2

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ docker run --rm ejemplo-combinado
Mensaje por defecto

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ docker run --rm ejemplo-combinado "Mensaje personalizado"
Mensaje personalizado

feijo@Valenfei MINGW64 ~/02-bash/WS2 (master)
$ |
```

Preguntas de análisis

1. ¿Cuál instrucción se comporta como valor por defecto?

CMD .

CMD proporciona el comando o los argumentos que se utilizarán por defecto cuando no se proporciona otro comando o argumento al ejecutar el contenedor.

2. ¿Cuál instrucción define el programa principal del contenedor?

ENTRYPOINT .

Por ejemplo:

```
ENTRYPOINT ["echo"]
```

hace que **echo** sea el ejecutable principal.

3. ¿Qué sucede con **CMD** cuando se pasa un comando al final de **docker run** ?

El comando proporcionado al final de **docker run** reemplaza el **CMD** .

Por ejemplo:

```
CMD ["echo","Hola desde CMD"]
```

y:

```
docker run --rm ejemplo-cmd "Mensaje reemplazado"
```

hace que se ejecute el comando proporcionado en lugar del `CMD`.

4. ¿Por qué `ENTRYPOINT` es útil cuando queremos que el contenedor se comporte como una herramienta concreta?

Porque permite establecer un **ejecutable principal fijo**.

Por ejemplo:

```
ENTRYPOINT ["echo"]
```

Entonces podemos ejecutar:

```
docker run --rm ejemplo-entrypoint "Hola"
```

y Docker utiliza:

```
echo Hola
```

Los argumentos pueden cambiar, pero el programa principal continúa siendo `echo`.

5. ¿Qué combinación utilizaría para crear un script que tenga un ejecutable fijo pero permita modificar parámetros?

Utilizaría:

```
ENTRYPOINT ["programa"] CMD ["parametro-por-defecto"]
```

La idea es:

```
ENTRYPOINT → ejecutable fijo  
CMD        → parámetros por defecto
```

Si no damos parámetros:

```
docker run mi-imagen
```

usa los parámetros de `CMD` .

Si proporcionamos otros:

```
docker run mi-imagen parametro-nuevo
```

se utilizan esos parámetros en lugar de los del `CMD` .

Tabla diferencias

Caso	Diferencia observada
CMD	Define un comando por defecto que puede ser reemplazado.
ENTRYPOINT	Define el ejecutable principal y recibe los argumentos de <code>docker run</code> .
ENTRYPOINT + CMD	Mantiene un ejecutable fijo y permite definir argumentos por defecto.